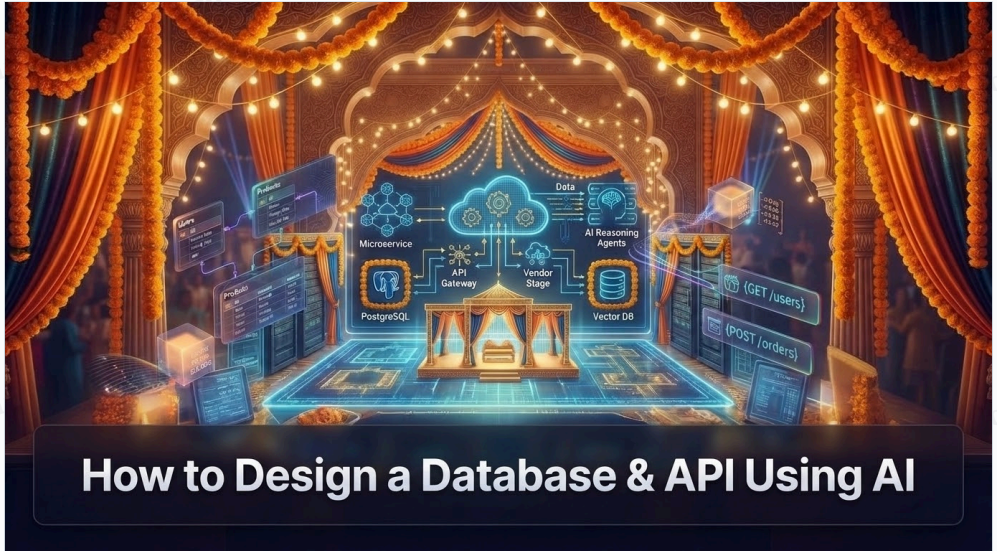


Database and API Design Documentation using AI



Till now, with the help of AI, we have collected and defined our PRD, and we have also defined our System Architecture, where we shortlisted the tech we will use.

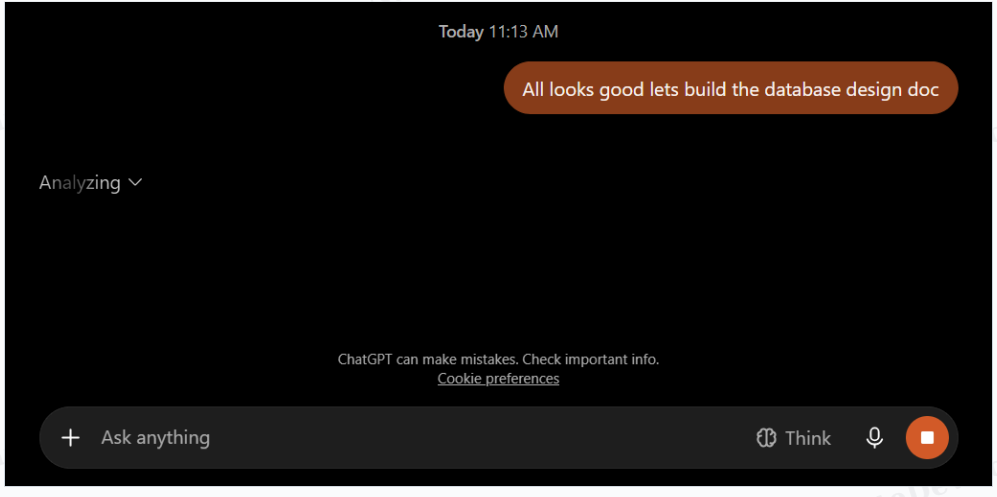
Here is the System Design Architecture: [Make My Marriage System Design](#)

Here is the PRD of the Project: [Make My Marriage PRD](#)

Now let's give a prompt without wasting any time. Let's build the database design.

I'm giving this prompt to my ChatGPT:

“All looks good, let's build the database design doc.”

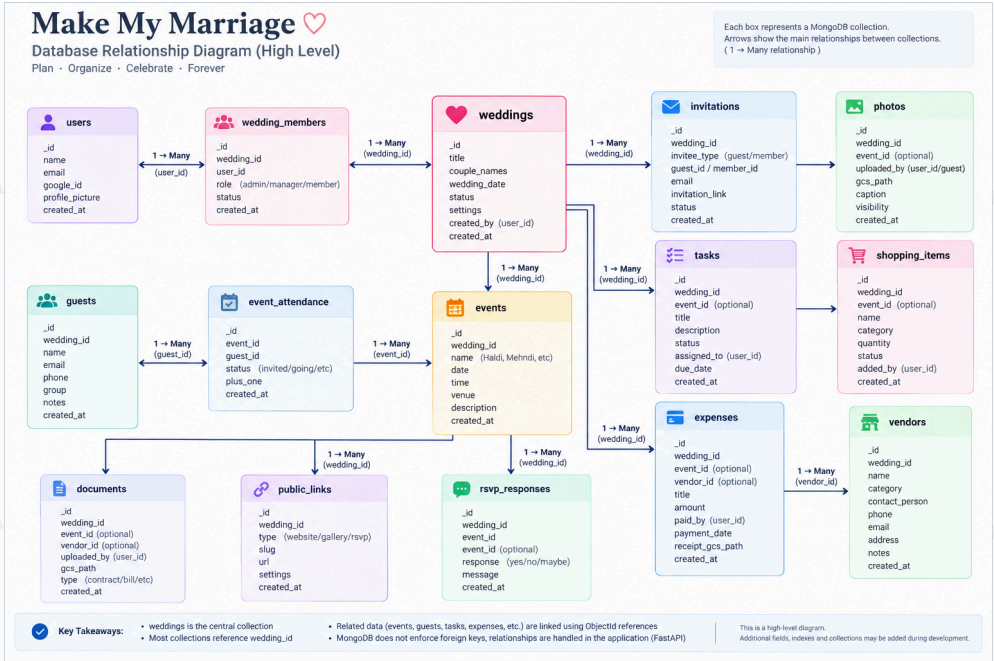


Below I have attached the generated Database Design document. You can see it below.

[Make_My_Marriage_Database_Design](#)

Database Design Document

A Database Design Document (DDD) is a formal blueprint or technical guide that outlines the structure, relationships, constraints, and organization of a database system. It translates high-level business requirements into a concrete technical plan for developers and database administrators.



There might be mistakes, and we will change this later if required, but this is just like V1.

Here, in the above image, you can see and identify the relationship, how we have attached the DB, and while development, you will get more clarity.

And coming to the design, I'm making it very si

The database is managed around the **Wedding** as the main/central record. When a user creates a wedding, a `wedding_id` is created, and most other data such as events, guests, tasks, shopping items, expenses, vendors, photos, and documents stores this `wedding_id` so we always know which wedding the data belongs to. Users are connected to a wedding through `wedding_members`, where we store their role like Admin, Manager, or Member. Guests are stored separately in the `guests` collection, and when a guest is invited to a particular event, we don't create another guest—we create an `event_attendance` record that connects the existing guest with that event and stores their RSVP/status. Similarly, tasks, expenses, shopping items, photos, and documents can optionally connect to a specific event using `event_id`. Vendors are stored once for the wedding and can be connected to multiple events through `vendor_event_links`. Photos and documents themselves are stored in **GCS**, while MongoDB only stores their file path and metadata. Invitations, RSVP responses, and public links are also connected to the wedding and, where required, to a guest or event. So basically, **Wedding is the main parent**, `wedding_id` connects the data to that wedding, and specific IDs like `event_id`, `guest_id`, `vendor_id`, and `user_id` create relationships between collections. FastAPI manages and validates these relationships and permissions; MongoDB stores the actual data.

Soft delete vs hard delete in DB

In a wedding events database, a **hard delete permanently erases data from the database, while a soft delete merely hides it by updating a status flag** (like setting an `is_deleted` column to true). For example, if a couple decides to completely cancel a "Bridal Shower" event and you **hard delete** it, that event and all guest RSVPs attached to it are gone forever and cannot be recovered. However, if a guest named "John Doe" cancels his attendance and you **soft delete** his record, his data remains safely in the database but is hidden from the active guest list; this allows the wedding planner to easily restore his RSVP if he changes his mind later, and preserves the database history without breaking the links between events and guests.

So we should never hard delete anything, and this is not good development practice.

But you can decide what to delete hard and what to soft delete, because at the end, it's your product and your expense, because the data you keep will cost you money to the provider.

Database Model Summary

The database is built around the **Wedding**. Each wedding gets a unique `wedding_id`, and this ID connects all the related data. **Users** are connected to weddings through `wedding_members`, where we store their role like Admin, Manager, or Member. **Guests** are stored once for the wedding, and `event_attendance` tells us which events each guest is attending. **Events** are connected to the wedding and can have their own tasks, expenses, vendors, and photos. **Tasks, shopping items, expenses, vendors, invitations, photos, and documents** are also connected to the wedding and, when needed, to a specific event. Photos and documents are stored in **GCS**, while MongoDB stores their details and file paths. Basically, **Wedding is the main parent, and IDs like `wedding_id`, `event_id`, `guest_id`, `user_id`, and `vendor_id` connect everything together**.

Now lets design the API doument

“This Looks really good, lets make a API design document now”

3. Authentication

Method	Endpoint	Purpose
GET	/auth/google	Start Google login
GET	/auth/google/callback	Handle Google OAuth callback
POST	/auth/logout	Logout
GET	/auth/me	Current user

4. Weddings & Members

Method	Endpoint	Purpose
POST	/weddings	Create wedding
GET	/weddings/{wedding_id}	Get wedding
PATCH	/weddings/{wedding_id}	Update wedding
GET	/weddings/{wedding_id}/members	List members
POST	/weddings/{wedding_id}/members/invite	Invite member by email + role
PATCH	/weddings/{wedding_id}/members/{member_id}	Change role/status
DELETE	/weddings/{wedding_id}/members/{member_id}	Remove member

Here is the API Design Document PDF [Make_My_Marriage_API_Design](#)

That's all for this.

This is called **Specification-Driven Development (SDD)** — where we first define the requirements and specifications clearly, and then use those specifications to guide the development.

I made you bored, huh? Next, we will work on some UI, and that will make you feel good.

Stay tuned, bye-bye! ☺



LEARN BY VIDEO

Explore courses on NamasteDev.com

Hands-on video lessons from Akshay Saini's ecosystem.

Start learning ↗